

VIAJE-ESTUDIOS:EVALUABLE

Instrucciones

Publica la aplicación de votación del viaje de estudios con Flask, Gunicorn y Nginx. Debes construir la imagen de la aplicación, crear una red de Docker y arrancar los dos contenedores mediante comandos.

Descarga 6-Viaje-estudios-votacion.zip desde los materiales. Incluye el código de Flask, wsgi.py, las dependencias y la plantilla HTML. Tendrás que crear el Dockerfile y la configuración de Nginx.

Antes de empezar: corrección de la plantilla

El ZIP original necesita el ajuste comunicado en clase: mueve la carpeta templates al interior de app, de modo que la plantilla quede en app/templates/index.html.

Para que el código busque la plantilla en esa ubicación, cambia en app.py la inicialización `Flask(__name__, template_folder="../templates")` por `Flask(__name__)`. Si ya habías construido la imagen, vuelve a construirla y crea de nuevo el contenedor después del cambio.

1. Crear el Dockerfile de la aplicación

Dentro de app/, prepara un Dockerfile con estos requisitos:

- Utiliza la imagen base `python:3.14.0-alpine3.22`.
- Establece /app como directorio de trabajo.
- Instala las dependencias de `requirements.txt` sin caché y copia el código, incluida la carpeta `templates`.
- Declara el puerto 8000.
- Arranca Gunicorn escuchando en `0.0.0.0:8000` y cargando `wsgi:app`. El comando de referencia es `gunicorn -b 0.0.0.0:8000 wsgi:app`.

2. Construir la imagen

Construye la imagen desde la carpeta app/ y asígnale el nombre viaje-estudios. Comprueba que la construcción termina correctamente.

3. Crear la red

Crea una red de Docker llamada webnet. Los dos contenedores deberán conectarse a ella para comunicarse por su nombre.

4. Arrancar Flask con Gunicorn

Ejecuta la imagen viaje-estudios en un contenedor llamado `fiesta_app`, conectado a webnet. El puerto 8000 se utilizará dentro de la red; el acceso desde el navegador se hará a través de Nginx.

Revisa `docker logs fiesta_app` y comprueba que Gunicorn está escuchando en el puerto indicado.

5. Configurar y arrancar Nginx

Crea el archivo `nginx/default.conf` con la configuración de un proxy inverso:

- Escucha en el puerto 80 y utiliza `server_name app.localhost;`.
- Reenvía las peticiones de `/` a `http://fiesta_app:8000`.
- Envía las cabeceras `Host`, `X-Real-IP`, `X-Forwarded-For` y `X-Forwarded-Proto`.

Arranca un contenedor llamado `nginx_proxy` con la imagen `nginx:1.27-alpine`, conectado a webnet. Publica el puerto 8080 del equipo hacia el 80 del contenedor y monta tu archivo de configuración en `/etc/nginx/conf.d/default.conf` en modo de solo lectura.

6. Comprobar el funcionamiento

- Abre `http://localhost:8080/` y comprueba que puedes votar y que se actualizan los resultados.
- Consulta `http://localhost:8080/health`: debe responder con `{"status": "ok"}`.
- Si aparece un error 502, revisa los nombres de los contenedores, su conexión a webnet y la dirección del proxy. Si la página no responde, revisa el mapeo `8080:80` y los logs.

Entrega

Adjunta el enlace a tu repositorio de GitHub con el proyecto y un `README.md` que incluya:

- Los comandos utilizados en los seis pasos, en el orden necesario para reproducir la práctica.
- Una captura de la aplicación funcionando en `http://localhost:8080/`.
- Una explicación de entre tres y cinco líneas sobre por qué el acceso público pasa por Nginx y cuál es su función delante de Flask y Gunicorn.

Evaluación

- Dockerfile y arranque con Gunicorn: **3 puntos**.
- Red de Docker y ejecución de los contenedores: **2 puntos**.
- Nginx como proxy inverso: **3 puntos**.
- README, captura y explicación: **2 puntos**.

Ampliación opcional, hasta 2 puntos: añade dos cabeceras de seguridad en Nginx, por ejemplo X-Content-Type-Options, X-Frame-Options o Referrer-Policy.

Materiales de la tarea

Los archivos adjuntos se descargan por separado desde la página de esta tarea.



6-Viaje-estudios-votacion.zip

Proyecto original de Flask · práctica con comandos de Docker

